Modern Programming Principles and Practice
Semester 2
Assessment #2

Module Title: **Modern Programming Principles and Practice**

Assessment Type: Practical Assessment – Individual

Weighting: **30%**

Maximal Possible Mark: 100 marks

Final Submission Date: AS PER MOODLE

Weekly Submission Dates start: AS PER MOODLE

Github Repo – oop-s2-2-mvc-<student-number> e.g. (oop-s2-2-mvc-1234)

Share To: JohnRowleyDorsetCollege

Assignment 2: "Food Safety Inspection Tracker"

Focus: Serilog logging, error handling, "audit trail" thinking, and building maintainable service-based code that will scale up to your main project.

Scenario

A local council tracks food premises inspections. Inspectors record inspection outcomes and follow-ups. Management wants visibility into what's happening and when things fail.

Required tech

- ASP.NET Core MVC
- EF Core + SQLite (or SQL Server)
- Identity roles
- **Serilog** logging (mandatory)
- xUnit tests (lighter than assignment 1 is fine)
- GitHub Actions CI

Entities

Use **3 entities**.

1. **Premises**
    - Id, Name, Address, Town, RiskRating (Low/Medium/High)
2. **Inspection**
    - Id, PremisesId, InspectionDate, Score (0–100), Outcome (Pass/Fail), Notes
3. **FollowUp**
    - Id, InspectionId, DueDate, Status (Open/Closed), ClosedDate (nullable)

Relationships:

- Premises 1—* Inspection
- Inspection 1—* FollowUp

Roles & access

- **Admin**: full access

- **Inspector**: can create inspections + follow-ups
- **Viewer** (or Manager): read-only dashboards (no create/edit)

Key features

A) Serilog logging (mandatory, assessed)

Set up Serilog with:

- Console sink
- Rolling file sink (daily)
- Enriched properties: Application, Environment, and UserName (from HttpContext when available)

Log at appropriate levels:

- Information: create/update actions (e.g., "Inspection created", include PremisesId, InspectionId)
- Warning: validation/business rule issues (e.g., creating FollowUp with due date before inspection date)
- Error: caught exceptions (include exception details)

Minimum requirement: at least **8 meaningful log events** across the app's main workflows.

B) Global error handling + "friendly failures"

- Use a global exception handler (or middleware) so unhandled exceptions show a friendly error page.
- Ensure exceptions are logged with Serilog.

C) A dashboard page with aggregations

Create a /Dashboard page:

- Count of inspections this month
- Count of failed inspections this month
- Count of open follow-ups overdue (DueDate < Today AND Status Open)
- Filter by Town and/or RiskRating

This forces them to write grouped queries and think about the "reporting" style they'll need in the main project.

D) Fake/seed data

Seed:

- 12 premises across 3 towns
- 25 inspections across different dates
- 10 follow-ups (some overdue, some closed)

E) CI workflow (repeat)

Same: build + test.

Minimum tests (xUnit)

At least **4 tests**, e.g.:

- Overdue follow-ups query returns correct items
- Follow-up cannot be closed without ClosedDate
- Dashboard counts consistent with known seed data (or use in-memory DB)
- Basic role authorization for Inspector vs Viewer (if feasible)

Marking)

- Serilog configured correctly + file output + enrichment: 30
- Logging coverage in workflows (not spam): 20
- Dashboard queries + filtering: 20
- EF Core model + migrations + seed data: 20
- CI passing + tests present: 10

Marking Rubric

Food Safety Inspection Tracker

Total: 100 Marks

Criterion 1: Serilog Configuration & Structured Logging (30 marks)

Level	Description	Marks
Excellent	Serilog correctly configured with console + rolling file sink. Structured logging used (e.g., IDs, user context). Appropriate log levels applied consistently. Logging integrated across main workflows.	26–30
Good	Serilog configured correctly but enrichment or structure partially applied.	18–25
Satisfactory	Basic logging implemented but limited structure or inconsistent log levels.	10–17
Poor	Logging missing, misconfigured, or not integrated meaningfully.	0–9

Criterion 2: Dashboard Queries & Filtering (20 marks)

Level	Description	Marks
Excellent	Correct aggregation queries (monthly inspections, failed inspections, overdue follow-ups). Filtering by Town/RiskRating works correctly. Efficient EF queries used.	17–20
Good	Dashboard mostly correct but minor logic or filtering gaps.	12–16
Satisfactory	Partial aggregation implemented or limited filtering.	6–11
Poor	Dashboard missing or incorrect.	0–5

Criterion 3: EF Core Model & Relationships (20 marks)

Level	Description	Marks
Excellent	Correct entity relationships (Premises → Inspection → FollowUp). Clean migrations. Seed data realistic and supports dashboard logic.	17–20
Good	Mostly correct design with minor configuration issues.	12–16
Satisfactory	Basic persistence but relational integrity partially weak.	6–11
Poor	Broken relationships or missing migrations.	0–5

Criterion 4: Error Handling & Logging of Failures (10 marks)

Level	Description	Marks
Excellent	Global exception handling implemented. Exceptions logged with context. Friendly error page displayed.	9–10
Good	Exceptions logged but error handling incomplete.	6–8
Satisfactory	Basic try/catch usage but inconsistent logging.	3–5
Poor	Unhandled exceptions or no meaningful logging of errors.	0–2

Criterion 5: Role-Based Access Control (10 marks)

Level	Description	Marks
Excellent	Clear separation between Admin, Inspector, and Viewer roles. Server-side authorization correctly enforced.	9–10
Good	Roles implemented but minor access-control gaps.	6–8
Satisfactory	Basic roles present but weak enforcement.	3–5
Poor	No meaningful authorization rules.	0–2

Criterion 6: GitHub & CI Workflow (10 marks)

Level	Description	Marks
Excellent	CI workflow builds and tests successfully in Release configuration. Repository cleanly structured.	9–10
Good	CI present but minor issues.	6–8
Satisfactory	Partial CI or inconsistent repository structure.	3–5
Poor	Missing or failing CI workflow.	0–2